PROGRAMMING IN JAVA

ASSIGNMENT 3 — CO3

EXCEPTION HANDLING AND MULTITHREADING

Student Details	**To be completed**
Name	J.SUBHAASHREE
Register Number	192424008
Course / Section	AI&DS
Faculty	Ravi Pujitha
Submission Date	5.11.26
GitHub Repository Link	https://github.com/Subhaashree06/JAVA-Assignments/blob/main/CO3_JavaAssignment3_Code.ipynb

ASSIGNMENT OVERVIEW AND SUBMISSION NOTES

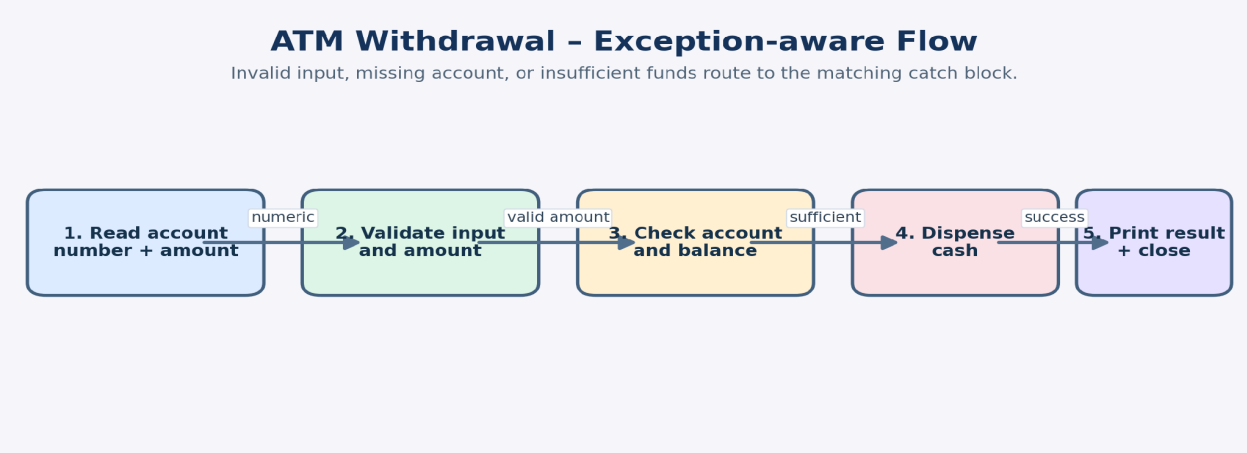
This report covers all seven tasks in Assignment 3 – CO3 (50 marks): exception handling, user-defined exceptions, resource safety, thread life cycle, synchronization, deadlock prevention, inter-thread communication and a concurrent fault-tolerant result processor.

Task	Topic	Marks	Main evidence included
Q1	ATM exception handling	6	Throwable hierarchy, failure table, test cases, stack trace, alternative error-code design
Q2	Custom exceptions and propagation	7	Exception class diagram, layered propagation, chaining code and test evidence
Q3	Resource safety and debugging	7	Manual-close vs try-with-resources code, leak explanation, suppressed exceptions
Q4	Thread creation and life cycle	7	Thread and Runnable implementations, life-cycle diagram, scheduler experiments
Q5	Cinema seat-booking race	6	Unsafe and synchronized code, critical section, alternative atomic approaches
Q6	Deadlock and producer-consumer	7	Deadlock example and prevention, TokenCounter complete code, bounded-buffer analysis
Q7	Concurrent result processor	10	Architecture diagram, pseudocode, full ExecutorService/Callable/Future implementation, measurement plan

Q1 HANDLING FAILURE IN A BANK ATM SIMULATION

Student A – Exception Design

The ATM can fail while reading user input, parsing an account number or amount, locating the card-reader file, validating a withdrawal amount, locating an account, checking funds, or dispensing cash. Java represents exceptional conditions under Throwable, whose two major branches are Error and Exception.



Relevant Throwable hierarchy

```
java.lang.Throwable
├── java.lang.Error                          (unchecked; serious JVM/system problems)
└── java.lang.Exception
    ├── java.io.IOException                  (checked)
    │   └── java.io.FileNotFoundException    (checked)
    └── java.lang.RuntimeException           (unchecked)
        ├── NumberFormatException
        ├── InputMismatchException
        ├── IllegalArgumentException
        └── IllegalStateException
```

Failure point	Exception / failure	Category	Where handled
Card-reader file missing	FileNotFoundException	Checked	catch(IOException)
Input contains letters where integer required	InputMismatchException	Unchecked	catch(InputMismatchException)
Malformed numeric text with parseInt	NumberFormatException	Unchecked	catch(NumberFormatException)
Negative or zero withdrawal	IllegalArgumentException (application validation)	Unchecked	catch(IllegalArgumentException)
Insufficient funds	InsufficientFundsException (custom)	Checked in this design	catch(InsufficientFundsException)
Account number not found	AccountNotFoundException (custom)	Checked in this design	catch(AccountNotFoundException)
JVM memory exhaustion	OutOfMemoryError	Error; not routine recovery	Not caught as normal business failure

Catch blocks must be ordered from the most specific type to the broadest type. For example, FileNotFoundException must precede IOException, and InputMismatchException must precede a general RuntimeException or Exception catch. If a broad catch is placed first, later narrower catches become unreachable and the compiler reports an error.

ATM implementation – try, multi-catch and finally

```
import java.io.*;
import java.util.*;

class InsufficientFundsException extends Exception {
    InsufficientFundsException(String message) { super(message); }
}
class AccountNotFoundException extends Exception {
    AccountNotFoundException(String message) { super(message); }
}
public class ATMSimulation {
    static Map<Integer, Double> accounts = new HashMap<>();
    static {
        accounts.put(1001, 5000.0);
        accounts.put(1002, 1200.0);
    }
    static void withdraw(int account, double amount)
            throws AccountNotFoundException, InsufficientFundsException {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        Double balance = accounts.get(account);
        if (balance == null) throw new AccountNotFoundException("Account not found");
        if (amount > balance)
            throw new InsufficientFundsException("Insufficient account balance");
        accounts.put(account, balance - amount);
        System.out.println("Please collect cash: Rs." + amount);
    }
    public static void main(String[] args) {
        System.out.println("ATM session started");
        try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Enter account number: ");
            int account = sc.nextInt();
            System.out.print("Enter withdrawal amount: ");
            double amount = sc.nextDouble();
            // Multi-catch for invalid numeric input types
            try {
                withdraw(account, amount);
            } catch (AccountNotFoundException | InsufficientFundsException e) {
                System.out.println("Transaction declined: " + e.getMessage());
            }
        } catch (InputMismatchException e) {
            System.out.println("Invalid input: enter numbers only.");
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid withdrawal: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("ATM service error: " + e.getMessage());
        } finally {
            System.out.println("ATM session closed. Please take your card.");
        }
    }
}
```

The finally block executes after the try/catch path in normal completion and ordinary exception handling, so it is suitable for session cleanup and status messages. It is not an absolute guarantee if the JVM halts, the process is forcibly terminated, or the machine loses power. The try-with-resources Scanner closes automatically.

Student B – Test inputs and trace table

Test	Input / condition	Expected path	finally executed?
1	1001, 500	Successful withdrawal; no catch	Yes
2	ABC, 500	InputMismatchException catch	Yes

3	1001, -100	IllegalArgumentException catch	Yes
4	1001, 7000	InsufficientFundsException catch in inner block	Yes
5	9999, 100	AccountNotFoundException catch in inner block	Yes
6	Card reader file absent in file-enabled version	FileNotFoundException / IOException catch	Yes

Illustrative uncaught stack trace and explanation

```
Exception in thread "main" java.lang.IllegalStateException: Card reader not initialized
    at ATMDebug.readCard(ATMDebug.java:18)
    at ATMDebug.main(ATMDebug.java:31)
```

Line 1 identifies the exception type and message. The first stack frame shows the method and source line where the exception was thrown. The next frame shows the caller that invoked that method. Stack frames are printed from the throw site outward through the call chain. If the exception is caught, the program normally prints no uncaught-exception trace unless printStackTrace() is called.

Demonstration of catch ordering: catch(Exception e) before catch(FileNotFoundException e) makes the second catch unreachable. The compiler rejects the code because FileNotFoundException is already covered by Exception.

Joint design question

Catching every exception with catch(Exception e) is acceptable only at a final application boundary where the program must log an unexpected failure, display a safe message and prevent a crash. It is not good as the only catch inside business logic: it hides the difference between invalid input, insufficient funds, missing files and programming defects. Specific catches allow correct recovery, useful messages and accurate logging.

Alternative concept – validation and error codes

```
static int withdrawWithoutExceptions(int account, int amount) {
    if (account <= 0) return -1;                 // invalid account input
    if (amount <= 0) return -2;                  // invalid amount
    if (!accounts.containsKey(account)) return -3;
    if (accounts.get(account) < amount) return -4;
    accounts.put(account, accounts.get(account) - amount);
    return 0;                                    // success
}
// Scanner validation: if (!sc.hasNextInt()) { print error; return; }
```

Pre-validation avoids using exceptions for expected invalid input and can be easier to follow when there are many routine validation failures. It requires explicit checks and error-code interpretation at each call. Exception construction and stack-trace capture have a relatively high constant cost and allocate stack-trace metadata; returning an integer status is usually cheaper. Both approaches require O(1) additional space for a fixed set of values; validating input is O(1) per field. Normal successful withdrawal work is O(1) for the HashMap average case.

Approach	Readability	Time / space
Exceptions	Clear separation for exceptional paths; meaningful exception types	Normal path low overhead; throwing/capturing stack trace is comparatively expensive; O(depth) trace storage
Pre-validation + error codes	More checks and code branches; caller must decode status	Usually cheaper for frequent expected invalid input; O(1) status storage

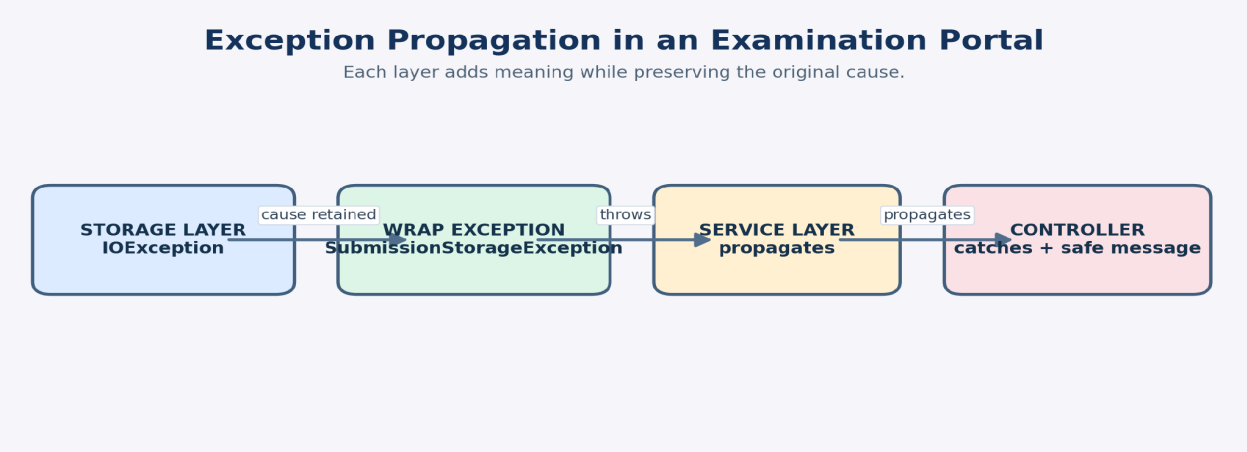
Q2 USER-DEFINED EXCEPTIONS AND PROPAGATION IN AN ONLINE EXAMINATION PORTAL

Student A – Exception hierarchy and propagation design

```
ExamException extends Exception                    [checked]
├── ExamWindowClosedException extends ExamException
├── StudentNotRegisteredException extends ExamException
└── SubmissionStorageException extends ExamException
    └── wraps IOException as its cause

InvalidSubmissionRequestException extends RuntimeException [unchecked]
```

ExamWindowClosedException, StudentNotRegisteredException and SubmissionStorageException are checked because the caller is expected to handle or explicitly propagate business/storage failures. InvalidSubmissionRequestException can be unchecked for programmer misuse or malformed internal API state that should be corrected rather than routinely recovered.



Layer responsibilities

Layer	Responsibility	Exception behaviour
Controller	Receives request, invokes service, converts known failure to student-safe response	Catches ExamException at the UI/API boundary
Service	Checks exam window and course registration; coordinates storage	Uses throw and declares throws ExamException
Storage	Writes file and translates low-level I/O failure	Catches IOException and throws SubmissionStorageException with cause

Completed TODO 1 – StorageDemo

```
import java.io.IOException;
class ExamException extends Exception {
    ExamException(String message) { super(message); }
    ExamException(String message, Throwable cause) { super(message, cause); }
}
class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String message, Throwable cause) { super(message, cause); }
}
class Storage {
    void save(String fileName, long sizeKb) throws SubmissionStorageException {
        try {
            if (sizeKb > 500) throw new IOException("Disk quota exceeded");
```

```
            System.out.println("Saved " + fileName);
        } catch (IOException e) {
            throw new SubmissionStorageException("Could not store " + fileName, e);
        }
    }
}
public class StorageDemo {
    public static void main(String[] args) {
        Storage storage = new Storage();
        try {
            storage.save("answer01.pdf", 120);
            storage.save("answer02.pdf", 800);
        } catch (ExamException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Cause: " + e.getCause().getMessage());
            e.printStackTrace();
        }
    }
}
```

Expected first two output lines: Saved answer01.pdf; Error: Could not store answer02.pdf; Cause: Disk quota exceeded. The full stack trace then shows the wrapped SubmissionStorageException and the original IOException as its cause.

Student B – failure scenarios and chaining evidence

Scenario	Exception	Observed handling
Exam closes at 5:00 PM; submission at 5:02 PM	ExamWindowClosedException	Service rejects submission; controller shows exam window closed
Student is not registered	StudentNotRegisteredException	Service rejects before storage is called
File exceeds 500 KB / disk quota failure	SubmissionStorageException caused by IOException	Storage wraps IOException; controller logs cause and displays safe message
Malformed internal request	InvalidSubmissionRequestException	Controller returns validation response; programming misuse is logged

getCause() returns the original IOException because it was passed to the SubmissionStorageException constructor. If the exception is re-thrown without the cause, getCause() returns null and the original low-level reason and stack trace are lost. In the supplied demo, the Cause line would print null if no cause were attached (or throw NullPointerException if code directly calls getCause().getMessage()).

Why meaningful storage exceptions are better

The UI should not need to understand file-system details such as disk quota, path permissions or operating-system I/O errors. SubmissionStorageException expresses the storage-layer meaning in the portal domain, preserves the original cause for diagnostics and lets the controller show a safe, consistent message.

Alternative concept – unchecked custom exceptions and complexity

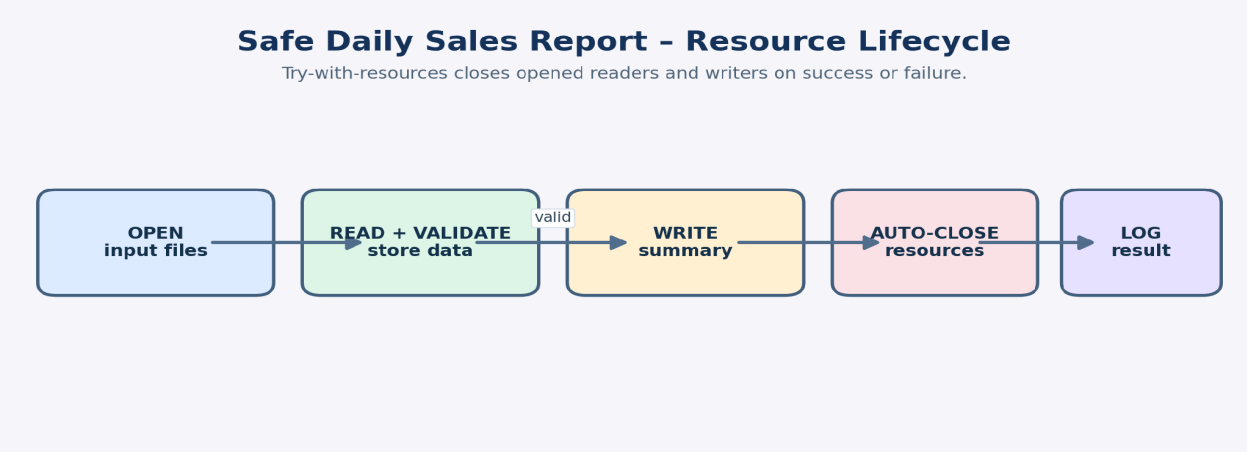
If ExamException extended RuntimeException, method declarations would not be required to include throws ExamException, and callers would not be forced by the compiler to catch or propagate it. This reduces signature noise but makes it easier for a caller to forget a required business failure. Stack-trace capture takes time proportional to stack depth and stores stack-frame information; chaining adds a reference to the cause and the cause's own trace/object graph. The exception object itself uses O(d) trace space for call depth d; a chain of k exceptions can retain O(total frames across the chain) diagnostic data.

Design	Method signature	Caller obligation
Checked	save(...) throws SubmissionStorageException	Must catch or declare it
Unchecked	save(...) (no throws required)	May catch, but compiler does not force handling

Q3 RESOURCE SAFETY, DEBUGGING AND ROBUST CODE

Shared implementation – report generation

The resources are each input BufferedReader/FileReader and the output BufferedWriter/FileWriter. Every successfully opened resource must be closed even if a later input file is missing, corrupt or throws while reading.



Manual try–catch–finally version

```
import java.io.*;
import java.util.*;

public class ManualSalesReport {
    public static void generate(String[] files, String output) {
        BufferedWriter writer = null;
        try {
            writer = new BufferedWriter(new FileWriter(output));
            long total = 0;
            for (String file : files) {
                BufferedReader reader = null;
                try {
                    reader = new BufferedReader(new FileReader(file));
                    String line;
                    while ((line = reader.readLine()) != null) {
                        total += Long.parseLong(line.trim());
                    }
                } catch (IOException | NumberFormatException e) {
                    System.err.println("Failed file " + file + ": " + e.getMessage());
                    throw new RuntimeException("Report incomplete", e);
                } finally {
                    if (reader != null) {
                        try { reader.close(); }
                        catch (IOException closeError) {
                            System.err.println("Close failed: " + closeError.getMessage());
                        }
                    }
                }
            }
            writer.write("TOTAL=" + total);
        } catch (IOException | RuntimeException e) {
            System.err.println("Report generation failed: " + e.getMessage());
        } finally {
            if (writer != null) {
                try { writer.close(); }
                catch (IOException e) { System.err.println("Output close failed: " +
e.getMessage()); }
```

```
                }
            }
        }
    }
```

A manual implementation can leak a resource if control exits through an exception before close() is reached, or if a close path is omitted. Nested try/finally is required for every resource. A common debugging test is to make the third store file missing or place a non-numeric line in the second file; set a breakpoint immediately after each open and inspect whether close() is reached.

Try-with-resources version

```
import java.io.*;
public class SafeSalesReport {
    public static void generate(String[] files, String output) {
        long total = 0;
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(output))) {
            for (String file : files) {
                System.out.println("Processing: " + file);
                try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
                    String line;
                    while ((line = reader.readLine()) != null)
                        total += Long.parseLong(line.trim());
                } catch (IOException | NumberFormatException e) {
                    System.err.println("Store file failed: " + file);
                    throw new RuntimeException("Could not process " + file, e);
                }
            }
            writer.write("TOTAL=" + total);
        } catch (IOException | RuntimeException e) {
            System.err.println("Report failed: " + e.getMessage());
        }
    }
}
```

Try-with-resources closes resources automatically when leaving the try block, whether normally or exceptionally. Multiple resources are closed in reverse order of their declaration. If the main operation throws and close() also throws, the close failure is recorded as a suppressed exception and can be retrieved using getSuppressed().

Student A – debugging and experimental analysis

Injected input	Manual close risk	Try-with-resources behaviour	Log evidence
All files valid	Normally closes if every path is correct	All readers and writer auto-close	All files processed
Second file missing	First reader closed if nested finally works; writer must still close	Reader/writer close automatically	Logs missing file name
Malformed line in third file	Exception may bypass close if close is not in finally	Current reader and writer auto-close	Logs corrupt file and line context
Close operation itself fails	Must handle close exception manually	Close exception is thrown or suppressed	Inspect getSuppressed()

Debugging steps: place breakpoints after FileReader construction, inside the read loop, in each finally block and at the outer catch. Step through a valid input and then a failing second/third input. Track how many readers are currently open. In the try-with-resources version, the count returns to zero when the method exits.

Student B – design and correctness

Try-with-resources is not sufficient by itself if the program writes a partially completed report and then treats it as valid, if a file is logically corrupt but syntactically readable, if a process crashes during disk flush, if files change while being read, or if the report requires transactional all-or-nothing publication. Use temporary output plus atomic rename, validation, checksums, retries where safe, logging, and explicit completeness status.

Suppressed exception demonstration

```
try (AutoCloseable resource = () -> {
        throw new IOException("close failed");
     }) {
    throw new IllegalStateException("processing failed");
} catch (Exception e) {
    System.out.println("Primary: " + e.getMessage());
    for (Throwable s : e.getSuppressed())
        System.out.println("Suppressed: " + s.getMessage());
}
```

Expected: Primary: processing failed; Suppressed: close failed. The primary failure remains the exception that caused the try block to exit, while close failure is retained rather than replacing it.

Alternative concept and complexity

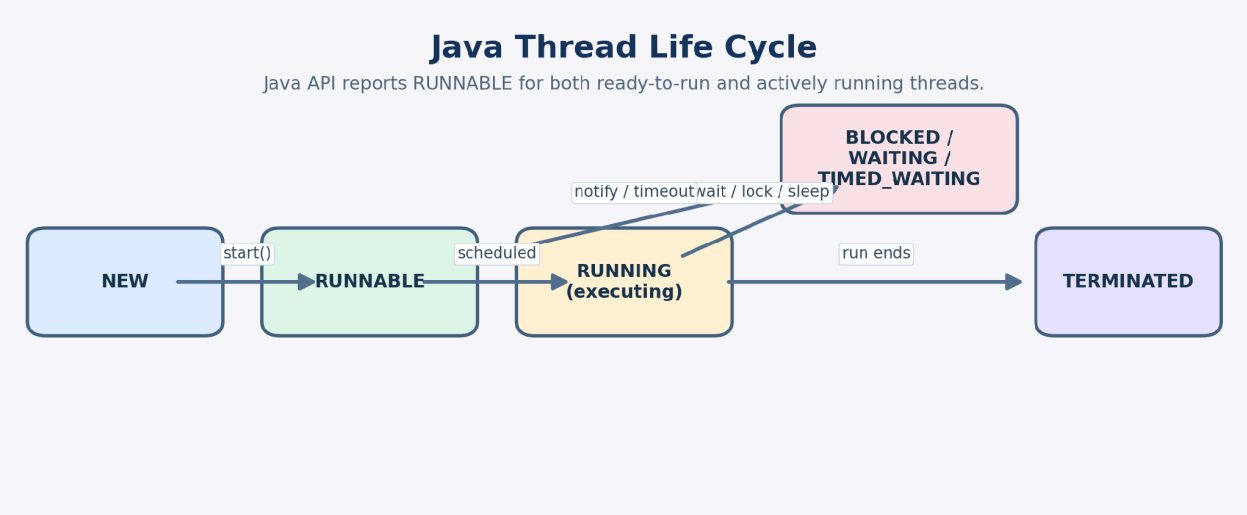
Approach	Time for f files and L total lines	Maximum open handles
Try-with-resources, one reader at a time	O(L) parsing plus file-open/close costs	At most 2 (one input reader + output writer)
Manual close in finally	O(L) if correctly implemented	At most 2 when nested cleanup is correct; can leak if cleanup path is omitted
No closing	O(L) until OS/JVM resource limit is reached	Can grow to O(f) open readers; may cause 'Too many open files'

Space for aggregation is O(1) if only a running total is retained; O(L) if all records are accumulated in memory. Try-with-resources is the least error-prone approach for deterministic resource cleanup.

Q4 — THREAD CREATION AND THE THREAD LIFE CYCLE

Student A – Thread design

Extending Thread is simple when the class itself represents a specialised thread, but it uses the single inheritance slot and mixes the task with the execution mechanism. Implementing Runnable separates the work from the thread and allows the task to be run by a Thread or an ExecutorService; it is usually preferred.



A Java thread starts in NEW. Calling start() moves it to RUNNABLE; the scheduler may run it. It can enter BLOCKED while waiting for a monitor, WAITING during wait()/join(), or TIMED_WAITING during sleep()/timed wait. When run() returns or terminates with an uncaught exception, it reaches TERMINATED.

Implementation – extending Thread

```
class ScannerStationThread extends Thread {
    private final int station, scripts;
    ScannerStationThread(int station, int scripts) {
        this.station = station; this.scripts = scripts;
    }
    public void run() {
        try {
            for (int i = 0; i < scripts; i++) Thread.sleep(10);
            System.out.println("Station " + station + " scanned " + scripts);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
public class ThreadStations {
    public static void main(String[] args) throws InterruptedException {
        Thread[] stations = new Thread[4];
        for (int i = 0; i < 4; i++) {
            stations[i] = new ScannerStationThread(i + 1, 5);
            stations[i].start();
        }
        for (Thread t : stations) t.join();
        System.out.println("All stations completed.");
    }
}
```

Implementation – implementing Runnable

```
class ScannerTask implements Runnable {
    private final int station, scripts;
    ScannerTask(int station, int scripts) {
        this.station = station; this.scripts = scripts;
    }
    public void run() {
        try {
            for (int i = 0; i < scripts; i++) Thread.sleep(10);
            System.out.println("Station " + station + " scanned " + scripts);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
public class RunnableStations {
    public static void main(String[] args) throws InterruptedException {
        Thread[] stations = new Thread[4];
        for (int i = 0; i < 4; i++) {
            stations[i] = new Thread(new ScannerTask(i + 1, 5));
            stations[i].start();
        }
        for (Thread t : stations) t.join();
        System.out.println("All stations completed.");
    }
}
```

Calling run() directly executes the method on the current thread (usually main); it does not ask the JVM to create or schedule a new thread. Calling start() creates a new execution path and then the JVM invokes run(). join() blocks the main thread until the target thread terminates, so the final total is printed only after every station completes.

Student B – behavioural experiments

Run	Possible finish order (illustrative only)	Observation
1	Station 2, 1, 4, 3	Order depends on scheduling
2	Station 1, 3, 2, 4	May differ from run 1
3	Station 4, 2, 1, 3	No fixed ordering guarantee
4–10	Record actual console order for each run	Paste actual output in final submission

Thread priorities are scheduling hints, not guarantees. Higher priority does not ensure earlier completion. Daemon threads do not keep the JVM alive; when all non-daemon threads (including main) finish, the JVM may exit while daemon work is still running. Never use daemon threads for tasks that must finish or persist important results.

The scheduler, processor core availability, operating-system scheduling, I/O timing and JVM runtime state can all change the observed output order. Correct programs must coordinate using join(), locks, queues or futures, rather than assuming a particular order.

Alternative – ExecutorService

```
ExecutorService pool = Executors.newFixedThreadPool(4);
try {
    List<Future<Integer>> results = new ArrayList<>();
    for (int station = 1; station <= 4; station++) {
        final int id = station;
        results.add(pool.submit(() -> {
```

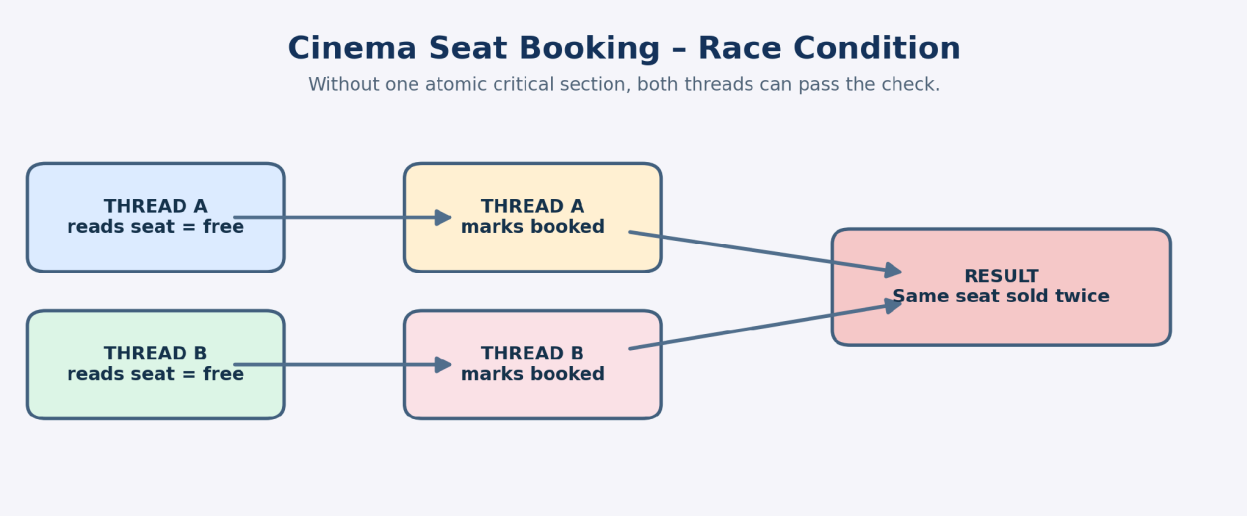
```
                int scanned = 5;
                Thread.sleep(50);
                System.out.println("Station " + id + " scanned " + scanned);
                return scanned;
            }));
        }
        int total = 0;
        for (Future<Integer> f : results) total += f.get();
        System.out.println("Total scanned: " + total);
    } finally {
        pool.shutdown();
    }
```

For n scripts and p worker threads, if each script takes approximately t time and work is balanced, ideal completion time is approximately ceil(n/p) × t plus scheduling and I/O overhead. Creating one Thread per station uses O(p) thread stacks. A fixed pool caps thread-stack use at O(p × stack size) and reuses workers. For four stations the fixed pool size can be four or smaller depending on CPU and I/O characteristics.

Q5 RACE CONDITIONS IN A CINEMA SEAT-BOOKING SYSTEM

Student A – producing and explaining the race

The unsafe operation is a check-then-act sequence: read booked[seat], observe false, then set booked[seat] = true and increment bookingsMade. These statements are not atomic as a group. Two threads can both observe false before either writes true.



Unsafe and synchronized implementations

```
public class SeatBooking {
    private final boolean[] booked;
    private int bookingsMade = 0;
    SeatBooking(int seats) { booked = new boolean[seats]; }

    // Unsafe version: race condition is possible.
    public boolean bookUnsafe(int seat) {
        if (!booked[seat]) {
            Thread.yield(); // widens the race window for demonstration
            booked[seat] = true;
            bookingsMade++;
            return true;
        }
        return false;
    }

    // Correct synchronized method: lock is this SeatBooking object.
    public synchronized boolean book(int seat) {
        if (booked[seat]) return false;
        booked[seat] = true;
        bookingsMade++;
        return true;
    }

    // Alternative synchronized block, locking only the critical section.
    public boolean bookWithBlock(int seat) {
        synchronized (this) {
            if (booked[seat]) return false;
            booked[seat] = true;
            bookingsMade++;
            return true;
        }
    }
```

```
    public static void main(String[] args) throws InterruptedException {
        SeatBooking screen = new SeatBooking(10);
        Thread[] users = new Thread[100];
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10;
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) t.join();
        System.out.println("Seats sold: " + screen.bookingsMade);
    }
}
```

Expected synchronized output: Seats sold: 10. The unsynchronized version may exceed 10 because multiple threads can pass the free-seat check before either marks the seat booked; each then increments the count. The exact number of duplicates is nondeterministic and depends on timing.

Student A – test design

Trial	Threads	Seats	Version	Expected observation
1–10	100	10	Unsafe	Record actual seats sold; race may produce >10
1–10	100	10	Synchronized method	Always 10
1–10	100	10	Synchronized block	Always 10

The exact critical section is the read of booked[seat], decision that it is free, write of true and increment of bookingsMade. All these operations must be protected by the same lock.

Student B – synchronization strategies

Strategy	Lock used	Advantages / limitations
synchronized method	Intrinsic monitor of this object	Simple; entire method locked, including validation and return path
synchronized block	Intrinsic monitor of chosen lock object	Can narrow critical section; must use the same lock for shared state
static synchronized	Class object monitor	Coordinates static shared state across instances; not required for one screen instance
ReentrantLock	Explicit lock instance	Supports tryLock, timeout and fairness options; unlock must be in finally

For a cinema with 20 screens, use one booking service/lock per screen or seat inventory, not one global lock for the entire cinema. This allows different screens to process bookings concurrently while ensuring a seat on one screen is protected. For higher throughput, use per-seat atomic state or database transactions with a unique seat constraint.

Alternative concepts

```
// AtomicBoolean per seat:
AtomicBoolean[] seats = new AtomicBoolean[seatCount];
for (int i=0; i<seatCount; i++) seats[i] = new AtomicBoolean(false);
boolean booked = seats[seat].compareAndSet(false, true);

// ReentrantLock:
private final ReentrantLock lock = new ReentrantLock();
boolean bookSeat(int seat) {
```

```
        lock.lock();
        try {
            if (booked[seat]) return false;
            booked[seat] = true; bookingsMade++; return true;
        } finally { lock.unlock(); }
    }

    // ConcurrentHashMap:
    boolean first = seatMap.putIfAbsent(seat, Boolean.TRUE) == null;
```

For s seats, AtomicBoolean[] uses O(s) extra state and compareAndSet is O(1) per attempt, with retries/contention possible. ReentrantLock uses O(1) lock metadata per lock (or O(s) if one lock per seat) and lock acquisition may block. ConcurrentHashMap uses O(s) entries and average O(1) putIfAbsent. All approaches require a consistent transaction boundary for the booking count and any payment/confirmation side effects.

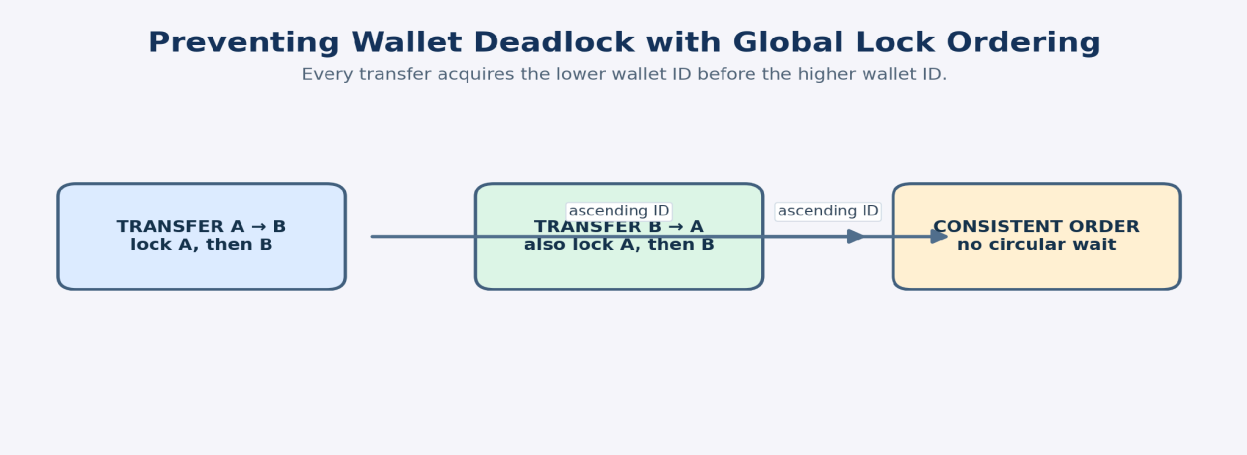
Q6 DEADLOCK AND INTER-THREAD COMMUNICATION

Part A – hostel wallet transfers

A deadlock can occur when Thread 1 locks wallet A and waits for B while Thread 2 locks wallet B and waits for A. The four Coffman conditions are mutual exclusion, hold-and-wait, no pre-emption and circular wait.

```
class Wallet {
    final int id;
    int balance;
    Wallet(int id, int balance) { this.id=id; this.balance=balance; }
}
class Transfers {
    // Demonstration of unsafe lock order (can deadlock).
    static void transferUnsafe(Wallet from, Wallet to, int amount) {
        synchronized (from) {
            try { Thread.sleep(20); } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); return;
            }
            synchronized (to) {
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }

    // Safe: acquire locks in ascending wallet-id order.
    static void transferSafe(Wallet from, Wallet to, int amount) {
        Wallet first = from.id < to.id ? from : to;
        Wallet second = from.id < to.id ? to : from;
        synchronized (first) {
            synchronized (second) {
                if (from.balance < amount) throw new IllegalArgumentException("Insufficient
balance");
                from.balance -= amount;
                to.balance += amount;
            }
        }
    }
}
```



A consistent global lock order removes circular wait: every transfer acquires the lower-ID wallet first and the higher-ID wallet second, regardless of transfer direction. Since all threads follow the same order, a cycle in the wait-for graph cannot form.

Part B – completed TokenCounter TODO 1 and TODO 2

```
public class TokenCounter {
    private final int[] buffer = new int[5];
    private int count = 0, in = 0, out = 0;

    public synchronized void put(int token) throws InterruptedException {
        while (count == buffer.length) wait();
        buffer[in] = token;
        in = (in + 1) % buffer.length;
        count++;
        notifyAll();
    }

    public synchronized int take() throws InterruptedException {
        while (count == 0) wait();
        int token = buffer[out];
        out = (out + 1) % buffer.length;
        count--;
        notifyAll();
        return token;
    }

    public static void main(String[] args) throws InterruptedException {
        TokenCounter counter = new TokenCounter();
        int[] collected = new int[2];
        Thread cook = new Thread(() -> {
            try {
                for (int t=1; t<=20; t++) counter.put(t);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        Thread[] students = new Thread[2];
        for (int s=0; s<2; s++) {
            final int id=s;
            students[s] = new Thread(() -> {
                try {
                    for (int k=0; k<10; k++) collected[id] += counter.take();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
        cook.start();
        for (Thread t : students) t.start();
        cook.join();
        for (Thread t : students) t.join();
        System.out.println("Sum of tokens collected: " +
            (collected[0]+collected[1]) + " (expected 210)");
    }
}
```

Expected output: Sum of tokens collected: 210 (expected 210). The counter is bounded: count is always between 0 and 5. The circular indexes in and out wrap around using modulo 5.

Student A – producer-consumer design

wait() must be called inside synchronized code because it releases and later reacquires the object's monitor. It must be in a while loop because a thread may wake spuriously, another thread may consume the condition before the awakened thread reacquires the lock, or multiple waiters may wake. The condition must be checked again.

notify() wakes one arbitrary waiter. With several producers and consumers, it may wake a thread whose condition is still false, leaving a suitable waiter asleep. notifyAll() wakes all waiters; each rechecks its condition under the monitor. sleep() pauses a thread but does not release a monitor it owns; wait() releases the monitor and waits for notification.

Student B – deadlock and prevention

Deadlock condition	How it appears
Mutual exclusion	Only one thread can own a wallet monitor at a time
Hold and wait	A thread holds the first wallet while waiting for the second
No pre-emption	A monitor cannot be forcibly taken away
Circular wait	T1 waits for B held by T2; T2 waits for A held by T1

What if while is changed to if?

Using if checks the condition only once. A spurious wakeup or a wakeup caused by a different operation can allow a producer to insert into a full buffer or a consumer to remove from an empty buffer. This can corrupt count/index state or cause invalid values. while is required for correctness.

Alternative concepts and complexity

Approach	Put/take or transfer cost	Space and reliability
wait/notifyAll bounded buffer	O(1) work per put/take, excluding wait time	O(5) buffer; correct but monitor protocol is easy to get wrong
ArrayBlockingQueue<Integer>(5)	O(1) average queue operations; put/take block when full/empty	O(5); safer library implementation with built-in condition handling
Ordered synchronized wallet locks	O(1) lock acquisition plus transfer work; contention may block	O(1) lock state per wallet; simple deadlock prevention
ReentrantLock.tryLock(timeout)	Bounded wait attempt; retry/backoff required	O(1) lock metadata; avoids indefinite wait but transfers may fail/retry

```
ArrayBlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
// Producer: queue.put(token);
// Consumer: int token = queue.take();

// Timeout-based transfer idea:
// if (firstLock.tryLock(100, TimeUnit.MILLISECONDS)) {
//   try { if (secondLock.tryLock(100, TimeUnit.MILLISECONDS)) {
//       try { performTransfer(); } finally { secondLock.unlock(); }
//   }} finally { firstLock.unlock(); }
// }
```

The queue abstraction is less error-prone for producer-consumer work because it encapsulates waiting, signalling and bounded capacity. Consistent lock ordering is simpler for wallet transfers; tryLock with timeout is useful when the application needs to avoid indefinite blocking and can safely retry.

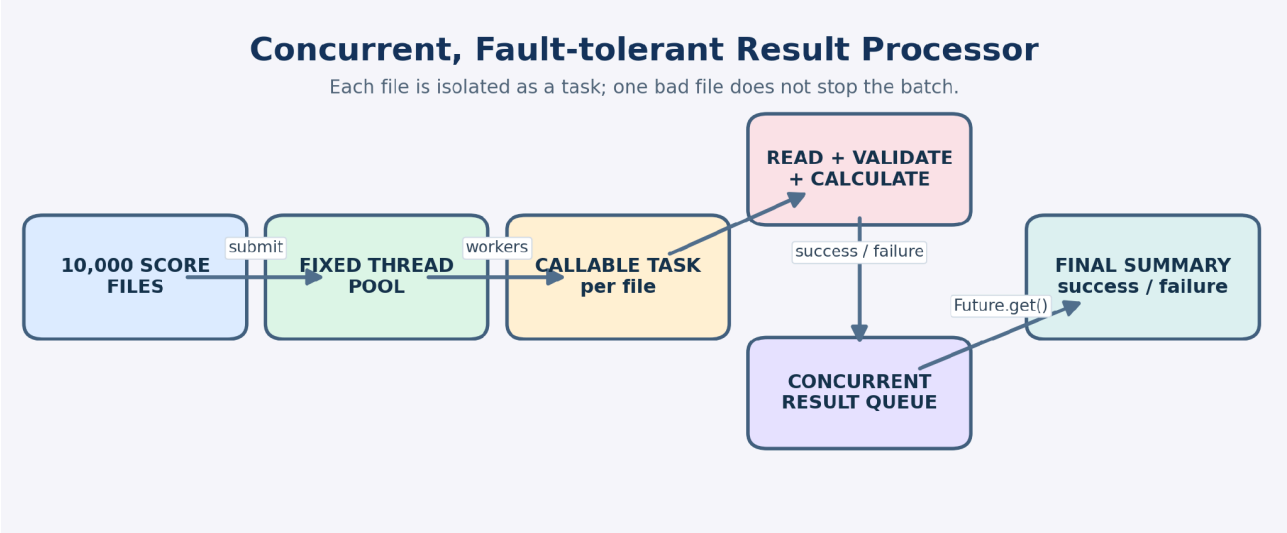
Q7 DESIGN A CONCURRENT, FAULT-TOLERANT PROCESSOR FROM FIRST PRINCIPLES

Stage 1 – Interpret the problem (Joint)

The system must process 10,000 independent score files. Each file can be read, validated and calculated independently, so file-level work can run in parallel. Shared state includes the aggregate success/failure counts and result collection. Failures include missing file, unreadable file, malformed score, invalid student identifier, out-of-range score and unexpected processing error.

Requirement	Measurable acceptance condition
Throughput	Process all 10,000 file paths with configurable pool sizes 1, 2, 4 and 8
Fault isolation	One bad file must not cancel other submitted tasks
Completeness	Exactly one terminal outcome (success or failure) per input file
Correctness	Success + failure count equals number of input files
Traceability	Each failure records file name and reason
Summary	Final summary is printed only after all submitted tasks complete

Stage 2 – Design (Student A)



Class design

Component	Responsibility
ResultProcessor	Discovers input files, creates executor, submits tasks and coordinates completion
ScoreTask implements Callable<FileResult>	Reads one file, validates fields and computes one result
FileResult	Immutable result record: file, status, student ID, total/grade or error message
ConcurrentLinkedQueue<FileResult>	Thread-safe collection for completed results
Future<FileResult>	Represents one task result and exposes get() / ExecutionException
Summary	Counts success/failure and writes a final report

Pseudocode

```
files = list all score files
pool = Executors.newFixedThreadPool(poolSize)
futures = []
for each file:
    futures.add(pool.submit(new ScoreTask(file)))
successes = 0; failures = 0
for each future:
    try:
        result = future.get()
        add result to concurrent results
        update success/failure count
    catch ExecutionException:
        record task failure for that file
shutdown pool and await completion
verify successes + failures == number of files
write final summary
```

The implementation uses a fixed thread pool to cap concurrency, Callable<FileResult> to return a value, Future.get() to collect completion, and ConcurrentLinkedQueue to store results safely. Each task catches expected file/data errors and returns a failure result rather than throwing an exception that could stop the entire run.

Stage 3 – Implementation and measurement (Student B)

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;

class FileResult {
    final String file, studentId, status, message;
    final int total;
    FileResult(String f, String id, String s, int t, String m) {
        file=f; studentId=id; status=s; total=t; message=m;
    }
    public String toString() {
        return file + " | " + status + " | " + studentId + " | " + total + " | " + message;
    }
}
class ScoreTask implements Callable<FileResult> {
    private final Path path;
    ScoreTask(Path path) { this.path=path; }
    public FileResult call() {
        try {
            List<String> lines=Files.readAllLines(path);
            if (lines.size() < 2) throw new IllegalArgumentException("Missing score data");
            String id=lines.get(0).trim();
            if (id.isEmpty()) throw new IllegalArgumentException("Missing student ID");
            int total=0;
            for (int i=1; i<lines.size(); i++) {
                int mark=Integer.parseInt(lines.get(i).trim());
                if (mark < 0 || mark > 100)
                    throw new IllegalArgumentException("Score outside 0..100");
                total += mark;
            }
            return new FileResult(path.getFileName().toString(),id,"SUCCESS",total,"validated");
        } catch (IOException | RuntimeException e) {
            return new FileResult(path.getFileName().toString(),"-","FAILURE",0,e.getMessage());
        }
    }
}
public class ConcurrentResultProcessor {
    static void run(Path folder, int poolSize) throws Exception {
        List<Path> files=new ArrayList<>();
```

```
        try (DirectoryStream<Path> stream=Files.newDirectoryStream(folder,"*.txt")) {
            for (Path p:stream) files.add(p);
        }
        ExecutorService pool=Executors.newFixedThreadPool(poolSize);
        ConcurrentLinkedQueue<FileResult> results=new ConcurrentLinkedQueue<>();
        List<Future<FileResult>> futures=new ArrayList<>();
        long start=System.nanoTime();
        try {
            for (Path p:files) futures.add(pool.submit(new ScoreTask(p)));
            for (Future<FileResult> f:futures) {
                try { results.add(f.get()); }
                catch (ExecutionException e) {
                    results.add(new FileResult("unknown","-","FAILURE",0,e.getCause().toString()));
                }
            }
        } finally {
            pool.shutdown();
            if (!pool.awaitTermination(1,TimeUnit.MINUTES)) pool.shutdownNow();
        }
        long elapsed=System.nanoTime()-start;
        long ok=results.stream().filter(r->r.status.equals("SUCCESS")).count();
        long bad=results.size()-ok;
        System.out.println("Files discovered: " + files.size());
        System.out.println("Success: " + ok);
        System.out.println("Failure: " + bad);
        System.out.println("Total outcomes: " + results.size());
        System.out.printf("Elapsed: %.3f ms (pool=%d)%n",elapsed/1e6,poolSize);
        if (results.size()!=files.size()) throw new IllegalStateException("Incomplete summary");
        try (BufferedWriter w=Files.newBufferedWriter(folder.resolve("summary.txt"))) {
            for (FileResult r:results) w.write(r.toString()+System.lineSeparator());
        }
    }
    public static void main(String[] args) throws Exception {
        Path folder=Paths.get(args.length>0?args[0]:"scores");
        int poolSize=args.length>1?Integer.parseInt(args[1]):4;
        run(folder,poolSize);
    }
}
```

Input file format used by this implementation: first line is student ID; each following line is one integer mark from 0 to 100. A valid file might contain STU001, 80, 75, 90. Inject a missing file by placing a path in a separate manifest or create an unreadable file; inject corrupt input with a non-numeric mark or a mark above 100. The current directory scan naturally discovers only files that exist, so to test missing files explicitly, use a manifest of expected filenames and submit every manifest entry as a task.

Small hand-checked test

File	Contents / condition	Expected outcome
S001.txt	STU001; 80; 75; 90	SUCCESS; total 245
S002.txt	STU002; 60; 70	SUCCESS; total 130
S003.txt	STU003; 80; BAD	FAILURE; malformed score
S004.txt	STU004; 120	FAILURE; score outside 0..100
S005.txt	missing path	FAILURE; file not found when manifest mode is used

For the five-case hand check: 2 successes + 3 failures = 5 outcomes. For the complete 10,000-file requirement, the invariant is success count + failure count = 10,000.

starting point; for I/O-heavy tasks, a somewhat larger pool may help, but measurements should decide.

Stage 4 – Defend the design (Joint)

The design guarantees a complete summary by submitting one task per expected input path, collecting one FileResult per task, waiting on every Future, and checking results.size() against the number of inputs before writing the summary. Each task converts expected per-file errors into a FAILURE result, so one corrupt file does not terminate the other tasks. A remaining limitation is that all result objects and Future objects are retained until the end; for very large datasets, a CompletionService or bounded producer-consumer pipeline could reduce memory use.

Alternative concept and complexity analysis

Approach	Expected time	Memory / failure handling
Fixed ExecutorService pool p	Approximately ceil(n/p) batches × per-file time, plus overhead	O(p × stack size) worker stacks + O(n) Future/result metadata; each task failure can be collected independently
parallelStream()	Uses common ForkJoinPool; speed depends on available parallelism and I/O behaviour	Uses shared pool; exception may escape terminal operation and affect the overall stream unless caught inside each file lambda
One thread per file	Can approach n-way parallelism only on idealized hardware; typically overwhelmed by scheduling	O(n × stack size) thread-stack reservation; expensive for 10,000 files; uncaught failure is isolated to that thread but aggregation still needs coordination

ExecutorService is recommended because it bounds resources, supports Callable/Future results, offers explicit shutdown and provides predictable failure collection. parallel streams are concise for simple CPU-bound transformations but provide less explicit pool and lifecycle control. One thread per file is not appropriate for 10,000 files because of thread creation, stack memory and scheduler overhead.

FINAL CONCLUSION

Assignment 3 demonstrates that robust Java applications require both controlled failure handling and safe concurrency. The ATM task uses specific exception handling and finally cleanup; the examination portal uses meaningful checked exceptions and exception chaining; report generation uses try-with-resources; scanner stations use Thread/Runnable and join; seat booking protects its critical section; wallet transfers use consistent lock ordering; the token counter uses wait/notifyAll with a bounded buffer; and the result processor uses ExecutorService, Callable, Future and a concurrent result collection.

GITHUB REPOSITORY STRUCTURE

```
CSA09-192424008-Assignment3/
├── README.md
├── Task01_ATM/
│   ├── ATMSimulation.java
│   └── outputs/
├── Task02_Exceptions/
│   ├── StorageDemo.java
│   └── outputs/
├── Task03_Resources/
│   ├── ManualSalesReport.java
│   ├── SafeSalesReport.java
│   └── test-files/
├── Task04_Threads/
│   ├── ThreadStations.java
│   ├── RunnableStations.java
│   └── outputs/
├── Task05_Synchronization/
│   ├── SeatBooking.java
│   └── outputs/
├── Task06_Deadlock_ProducerConsumer/
│   ├── Transfers.java
│   ├── TokenCounter.java
│   └── outputs/
└── Task07_ConcurrentProcessor/
    ├── ConcurrentResultProcessor.java
    ├── scores/
    └── measurements/
```

ADDITIONAL EXECUTION EVIDENCE AND REQUIRED EXPERIMENT SNIPPETS

The following short additions make the run-time observations requested in the assignment straightforward to collect. Include the resulting console output and screenshots in the matching GitHub task folders.

Q4 – isAlive(), priority and daemon demonstrations

```
// After starting each station thread:
for (Thread t : stations)
    System.out.println(t.getName() + " alive? " + t.isAlive());

// Priority experiment (priority is only a scheduling hint):
stations[0].setPriority(Thread.MAX_PRIORITY);
stations[1].setPriority(Thread.MIN_PRIORITY);

// Daemon experiment: set before start().
Thread background = new Thread(() -> {
    try { Thread.sleep(5000); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    System.out.println("Daemon task finished");
});
background.setDaemon(true);
background.start();
// If main and all non-daemon threads finish first, the JVM may exit
// before the daemon prints its completion message.
```

Run the priority experiment at least ten times and record actual finish order. Do not claim that the maximum-priority thread must finish first. Run the daemon experiment separately because the JVM may terminate before the daemon task completes.

Q5 – repeatable race and timing harness

```
// Run the following logic separately for unsafe, synchronized-method,
// and synchronized-block versions. Construct a fresh screen per trial.
for (int trial = 1; trial <= 10; trial++) {
    SeatBooking screen = new SeatBooking(10);
    Thread[] users = new Thread[100];
    long start = System.nanoTime();
    for (int i = 0; i < users.length; i++) {
        final int seat = i % 10;
        users[i] = new Thread(() -> screen.bookUnsafe(seat)); // replace method per version
        users[i].start();
    }
    for (Thread t : users) t.join();
    long elapsed = System.nanoTime() - start;
    System.out.println("Trial " + trial + ": sold=" + screen.bookingsMade
        + ", elapsedMs=" + elapsed / 1_000_000.0);
}
```

For a fair comparison, run the same number of threads, seat distribution and JVM conditions for each implementation. The synchronized versions must never exceed ten sales. The unsafe result is timing-dependent; a run may occasionally show ten even though the implementation is incorrect.

Q3 – debugging checklist for the manual resource version

- Set a breakpoint immediately after opening the output writer.
- Set a breakpoint immediately after opening each store reader.
- Run once with all valid files and verify each reader reaches its close() call.

• Run again with the second or third file missing, or with a non-numeric line.

• Step through the catch and finally blocks; record the open-reader count after method exit.

• Repeat with try-with-resources and verify that all successfully opened resources are closed.

Q7 – manifest-based missing-file test

```
// Use a manifest containing expected filenames, including absent files.
List<Path> expected = Files.readAllLines(folder.resolve("manifest.txt"))
    .stream().map(String::trim).filter(s -> !s.isEmpty())
    .map(folder::resolve).collect(Collectors.toList());

// Submit one ScoreTask for EVERY expected path, not only existing files.
// Files.readAllLines(path) inside ScoreTask will then return a failure result
// for a missing file, ensuring the final outcome count equals manifest size.
```

This modification is important: directory scanning alone cannot report a file that is missing because the absent file is not discovered. A manifest defines the expected 10,000 inputs and makes missing-file accounting possible.

COLOUR DIAGRAM – ATM THROWABLE HIERARCHY

